Project 1: Streaming Analytics Pipeline on Azure PostgreSQL

Domain: CareVault remote patient monitoring

Big data engineering plan, version 2 (September 2026)

Item	Detail
Course	Big Data, Team Project 1. Deliverables i to vi from the assignment brief.
Domain	Wearable vitals (CGM, BP cuff, smartwatch) streaming into CareVault for patients with diabetes and hypertension
Core stack	Azure Database for PostgreSQL Flexible Server 18, pg_partman, pg_cron, Python 3.13, psycopg 3, River, Streamlit 1.63
What sets it apart	Measured scale (step-load to saturation), effectively-once processing, clinical-grade evaluation, full reproducibility
Planning assumptions	4 members, 5 weeks. Adjust Sections 14 and 15 to your real team and deadline.

1. The Quality Bar

Most teams will meet the six deliverables. What separates an exceptional project is not the number of tools, it is evidence. Every claim in our report should be backed by a number we measured, a design choice we can defend, and a result someone else can reproduce with one command. The targets below are our internal bar. We set the final values after the week 2 baseline, and we report honestly where we fall short.

Dimension	Target	How we prove it
Sustained ingest	>= 50,000 rows/min on the benchmark tier; find the saturation point	Step-load curve, throughput vs latency
End-to-end latency	p95 < 3 s from event to alert/prediction at 20k rows/min	event_time vs processed_at, percentile table
Data volume	>= 20 million readings at final run	Row counts, table and index sizes
Dashboard queries	p95 < 300 ms at full volume	pg_stat_statements, EXPLAIN ANALYZE
Processing semantics	No lost or double-counted readings under restarts	Kill-and-restart test, reconciliation query
Outlier detection	F1 >= 0.90 on sensor errors, report per type	Confusion matrix vs injected ground truth
Online ML	Beat persistence baseline MAE by >= 20%, >= 99% in Clarke zones A+B	Prequential curves over 3 seeds
Drift	Detect injected drift and recover error within a stated window	ADWIN detection delay, recovery time
Reproducibility	Fresh clone to running system in one command	README, Makefile, CI badge

2. Summary and Scope

We simulate the streaming side of CareVault. Each patient wears a monitoring kit that reports six vitals every minute. Generators produce tens of thousands of readings per minute into Azure PostgreSQL. Parallel processor workers detect outliers, separate sensor faults from real clinical events, and run online models that forecast glucose 15 minutes ahead while learning continuously. A Streamlit dashboard shows clinical, model and system health in near real time.

In scope	Out of scope (future work)
Partitioned schema on Azure PostgreSQL 18 with 8 related tables	OCR, document upload, LLM extraction
Parallel generators with realistic physiology and messy data	FHIR, EHR integration, user authentication
Effectively-once micro-batch processing with horizontal scaling	Kafka, Spark (justified in Section 3.3)
Three-layer outlier detection with ground-truth evaluation	Mobile app, emergency card, billing
Online glucose forecasting with drift handling	Real patient data (synthetic only, no PHI)
Benchmark harness, CI, one-command setup	Multi-region deployment

2.1 Requirement mapping

#	Requirement	Our implementation	Evidence
i	3+ related tables on Azure PostgreSQL	8 tables, FKs, daily partitions, rollup table	ER diagram, portal screenshot
ii	Hundreds of rows per minute	Tens of thousands per minute, measured to saturation	Step-load chart
iii	Live dashboard	Streamlit, 5 pages, fragment refresh every 5 s	Live demo
iv	Outlier detection	Rules, rolling MAD, Half-Space Trees	P/R/F1 per anomaly type
v	ML on the stream	River models, prequential, delayed labels, drift	MAE curves, Clarke grid
vi	Report and 5-minute video	Sections 13 outlines	Final PDF, recording

3. Technology Stack

3.1 Versions (checked September 2026)

Layer	Choice	Why
Database	Azure Database for PostgreSQL Flexible Server, PostgreSQL 18 (GA on Azure)	Latest major version. Faster COPY and better partition handling. Required by brief.
Partitioning	Native declarative partitioning managed by pg_partman, scheduled by pg_cron	Both supported on Azure Flexible Server. Instant retention by dropping partitions.
Observability	pg_stat_statements, Azure Monitor metrics	Query-level timing for the report
Language	Python 3.13, uv for environment and lockfile	Fast, reproducible installs. Pin exact versions in uv.lock on day 1.
DB driver	psycopg 3 with COPY and pipeline mode	Highest-throughput Python path into PostgreSQL
Online ML	River 0.24.x	Purpose-built streaming ML, drift detectors, Half-Space Trees
Dashboard	Streamlit 1.63 with st.fragment(run_every, parallel=True)	Independent live panels, no full-page reruns
Dev and CI	Docker Compose (local PostgreSQL 18 mirror), GitHub Actions, pytest, ruff	Same schema locally and on Azure, tests on every push
Provisioning	Azure CLI script (optionally Bicep)	Server, parameters and firewall rules as code

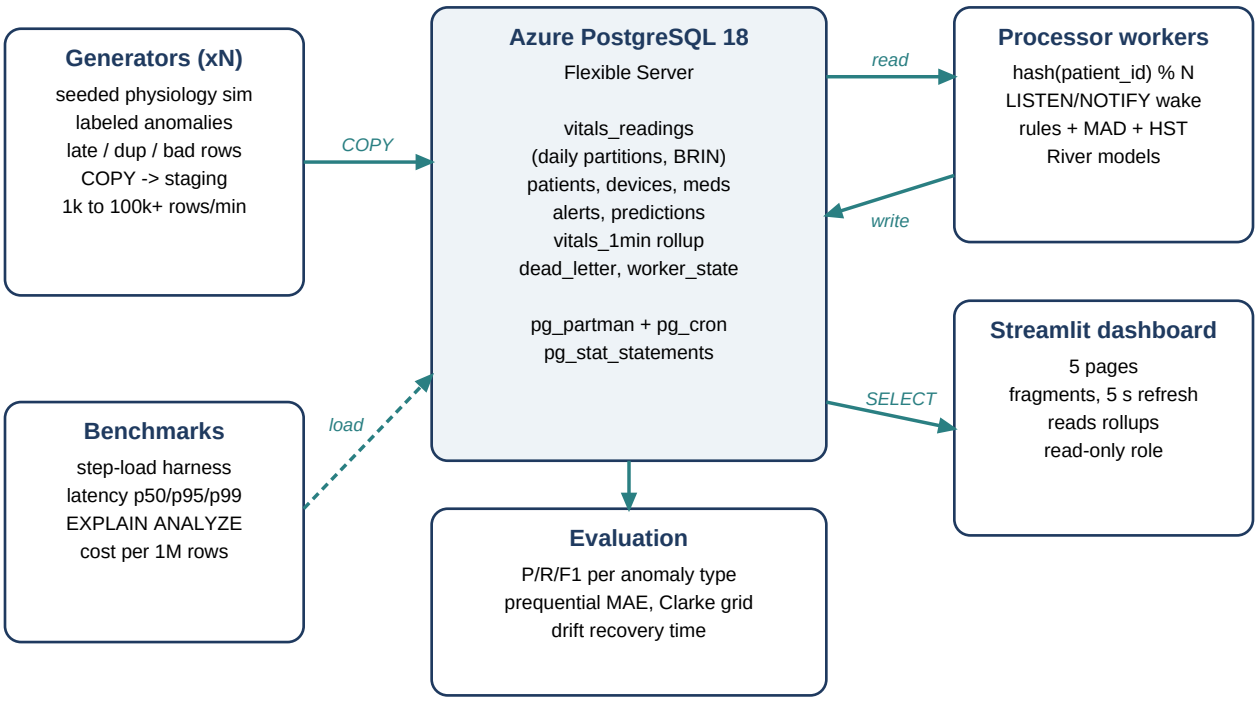
3.2 Why not TimescaleDB

TimescaleDB would give us hypertables for free, but Azure provides only its Apache-2 edition, which excludes features like native compression and continuous aggregates, and extension support differs between major versions. Native partitioning plus pg_partman is fully supported on PostgreSQL 18 on Azure, and building the rollup ourselves is a stronger demonstration of big data engineering. Check the extension list in the portal on day 1; if TimescaleDB is available for PG 18, we can add a one-page comparison as a stretch goal.

3.3 Why no Kafka or Spark

The brief's reference architecture writes straight into PostgreSQL. We treat PostgreSQL as the durable log: generators append, workers track their position, and all writes are transactional. This gives us the guarantees Kafka would provide at our scale without extra infrastructure. The report states the crossover point where a broker would become worthwhile, using our own saturation numbers as evidence. That is a stronger argument than adding Kafka by default.

4. System Architecture



- **Generators** run as N parallel processes, each owning a shard of patients. They COPY batches into an unlogged staging table, then move rows into the partitioned table with deduplication in a single statement.
- **PostgreSQL** stores raw readings in daily partitions. pg_cron runs partition maintenance and refreshes a per-minute rollup table.
- **Processor workers** each own the patients where hash(patient_id) mod N equals their index. They wake on NOTIFY (with a 2-second polling fallback), process a micro-batch, and commit results and

their progress in one transaction.

- **Dashboard** reads through a read-only role, mostly from the rollup table, so its speed does not degrade as raw data grows.
- **Benchmark harness** drives the generators through load steps and records throughput, latency and database metrics.

5. Database Design

5.1 Tables

Table	Purpose	Notes	Growth
patients	Monitored people	Condition, baselines	Static, 2,000
devices	Wearable kit	Firmware version (one version is buggy)	Static, 2,000
medications	Active drugs	Drives generator effects and ML features	Static, ~3,500
vitals_readings	The stream	Partitioned daily on recorded_at, BRIN index	Tens of thousands/min
vitals_1min	Rollup per patient per minute	Upserted by pg_cron each minute	~1 row per patient-minute
alerts	Outlier output	FK to reading, sensor_error or clinical	~1% of readings
predictions	Model output	Partitioned daily, filled when label arrives	Same as readings
dead_letter	Rejected rows	Malformed or impossible payloads with reason	~0.2% of readings
worker_state	Processing progress	One row per worker	N rows

5.2 Key design decisions

- **Composite primary key.** On a partitioned table the primary key must include the partition column, so it is (reading_id, recorded_at). Alerts and predictions reference both columns.
- **Event time vs ingest time.** `recorded_at` is when the device measured; `ingested_at` is when the row landed. Latency and late-arrival analysis depend on having both.
- **Natural key for idempotency.** A unique constraint on (device_id, recorded_at) makes duplicate device retries harmless.
- **BRIN on recorded_at.** Readings arrive in time order, so a BRIN index is a fraction of the size of a B-tree while still pruning blocks. We measure both and report the difference.
- **PostgreSQL 18 virtual generated column.** Mean arterial pressure `map` is computed on read, costing no storage. If pg_partman templating rejects it, fall back to a view.
- **Retention.** Raw partitions older than 45 days are dropped automatically; rollups are kept. Dropping a partition is instant, unlike DELETE.

5.3 DDL (core)

```
CREATE TABLE vitals_readings (
  reading_id    BIGINT GENERATED ALWAYS AS IDENTITY,
  patient_id    INT NOT NULL REFERENCES patients(patient_id),
  device_id     INT NOT NULL REFERENCES devices(device_id),
  recorded_at   TIMESTAMPTZ NOT NULL,                  -- event time from device
  ingested_at   TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
  heart_rate REAL, systolic_bp REAL, diastolic_bp REAL,
  glucose_mgdl REAL, spo2 REAL, temp_c REAL,
  map REAL GENERATED ALWAYS AS ((systolic_bp + 2 * diastolic_bp) / 3) VIRTUAL,  -- PG18
  injected_type TEXT,                                  -- ground truth, NULL if normal
  PRIMARY KEY (reading_id, recorded_at),
  UNIQUE (device_id, recorded_at)                      -- dedupe device retries
) PARTITION BY RANGE (recorded_at);

CREATE INDEX vr_brin ON vitals_readings USING brin (recorded_at);
CREATE INDEX vr_patient_time ON vitals_readings (patient_id, recorded_at DESC);

CREATE TABLE alerts (
  alert_id     BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  reading_id   BIGINT NOT NULL,
  recorded_at  TIMESTAMPTZ NOT NULL,
  patient_id   INT NOT NULL REFERENCES patients(patient_id),
  metric TEXT NOT NULL, value REAL, score REAL, detector TEXT NOT NULL,
  alert_type   TEXT NOT NULL CHECK (alert_type IN ('sensor_error', 'clinical')),
  severity     TEXT NOT NULL CHECK (severity IN ('low', 'high')),
  detected_at  TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
  FOREIGN KEY (reading_id, recorded_at) REFERENCES vitals_readings (reading_id, recorded_at),
  UNIQUE (reading_id, metric, detector)                -- idempotent reprocessing
);

CREATE TABLE vitals_1min (
  patient_id INT NOT NULL REFERENCES patients(patient_id),
  minute     TIMESTAMPTZ NOT NULL,
  n INT, glucose_avg REAL, glucose_min REAL, glucose_max REAL, sbp_avg REAL, hr_avg REAL,
  PRIMARY KEY (patient_id, minute)
);

SELECT partman.create_parent(p_parent_table => 'public.vitals_readings',
  p_control => 'recorded_at', p_interval => '1 day', p_premake => 7);
UPDATE partman.part_config SET retention = '45 days', retention_keep_table = false
  WHERE parent_table = 'public.vitals_readings';
SELECT cron.schedule('partman', '*/30 * * * *', $$CALL partman.run_maintenance_proc()$$);
```

patients, devices, medications, predictions (partitioned the same way), dead_letter and worker_state follow the same conventions and live in `sql/schema.sql`.

5.4 Azure setup

- PostgreSQL 18, Burstable B2s for development. Scale compute up to a General Purpose tier only for benchmark sessions, then scale back down. Record cost per session for the report.
- Server parameters: add pg_partman, pg_cron and pg_stat_statements to `azure.extensions`; add pg_cron and pg_stat_statements to `shared_preload_libraries`; set `cron.database_name` to carevault. Restart once.
- Roles: `pipeline_rw` for generators and workers, `dashboard_ro` read-only. Public access with per-member firewall rules, SSL required.
- Secrets live in .env (git-ignored). Set a budget alert on the subscription on day 1.

6. Ingestion at Scale

6.1 Write path

Row-by-row INSERT tops out quickly. Each generator batches readings, streams them with COPY into an unlogged staging table (no WAL cost), then moves them into the partitioned table in one statement that drops duplicates and routes invalid rows to dead_letter.

```
def flush(conn, rows):
    with conn.transaction(), conn.cursor() as cur:
        with cur.copy(f"COPY staging_vitals ({COLS}) FROM STDIN") as cp:
            for r in rows:
                cp.write_row(r)  # one network round trip per batch
        cur.execute(f"INSERT INTO vitals_readings ({COLS}) SELECT {COLS} FROM staging_vitals "
                    f"WHERE {VALID} ON CONFLICT (device_id, recorded_at) DO NOTHING")
        cur.execute(f"INSERT INTO dead_letter SELECT now(), 'invalid', to_jsonb(s) FROM staging_vitals s WHERE NOT
        cur.execute("TRUNCATE staging_vitals; NOTIFY new_readings")  # wake workers
```

Each generator process uses its own staging table (staging_vitals_1, _2, ...) so parallel writers never block each other.

6.2 Realistic messiness (the veracity part of big data)

Case	Rate	What we expect the system to do
Duplicate retries	0.5%	Silently dropped by the unique key; counted in metrics
Late arrivals (1 to 10 min)	2%	Land in the correct partition; rollup re-aggregates the last 15 minutes
Out-of-order within a batch	5%	Workers sort by event time per patient before processing
Malformed payloads	0.2%	Routed to dead_letter with a reason
Device offline gaps	Random	Missing-data rate shown on dashboard; model skips gaps

6.3 Backfill then go live

Before the demo we backfill 30 simulated days at maximum speed to reach 20 million+ rows, then switch to real-time mode. This gives the models history to warm up on and makes every query in the report run at realistic volume.

7. Stream Processing

7.1 Effectively-once semantics

A common hidden bug: tracking progress by the highest reading_id seen. Identity values are assigned at insert, but transactions can commit out of order, so a row with a smaller id can become visible after a larger one was already processed. It would be skipped forever.

Our fix: each worker reads rows by ingest time with a 10-second overlap window, keeps a small set of recently processed reading_ids to skip repeats, and writes alerts, predictions and its new position in

one transaction. Reads are at-least-once, writes are idempotent (unique constraints), so the outcome is effectively-once. We prove it with a kill-and-restart test: stop a worker mid-batch, restart it, and run a reconciliation query showing every reading has exactly one prediction.

```
with conn.transaction():
    cur.executemany(ALERT_SQL, alerts)        # ON CONFLICT DO NOTHING
    cur.executemany(PRED_SQL, preds)          # ON CONFLICT DO NOTHING
    cur.execute("UPDATE worker_state SET last_ingested_at = %s WHERE worker = %s", (hi, wid))
```

7.2 Horizontal scaling

Worker k owns patients where hash(patient_id) mod N = k, so per-patient state (MAD windows, model buffers) never has to be shared. We benchmark 1, 2 and 4 workers and plot throughput and lag. Online model weights are shared by periodically merging to a global model snapshot, or kept per worker if merging is not worth it; we report which works better.

8. Data Generator

Seeded and deterministic, so any run can be reproduced exactly. Each patient has baselines drawn from their condition, plus daily rhythm, meals, medication effects and noise.

Vital	Model
Glucose	Baseline 95 to 160 by condition. Meal spikes of 30 to 80 decaying over ~2 h. Insulin and metformin pull toward baseline. Noise sd 6.
Blood pressure	Baseline 115/75 to 145/92. Morning surge. ACE inhibitor or beta blocker lowers 8 to 12.
Heart rate	62 to 80 with daily rhythm and activity bursts. Beta blocker lowers ~8.
SpO2, temperature	Stable with small noise. Main targets for sensor-error detection.

8.1 Injected anomalies (ground truth)

Type	Share	Behaviour	Correct label
spike	30%	Single extreme value on one metric	sensor_error
dropout	20%	SpO2 or temperature reads 0	sensor_error
firmware_bug	15%	Firmware v2.3 adds +40 glucose for 10-minute runs	sensor_error
stuck_sensor	5%	Same value repeated for 15+ readings	sensor_error
hypoglycemia	15%	Gradual fall to 45 to 60 with HR rise	clinical
hypertensive	15%	Systolic climbs to 180 to 200 over several readings	clinical

Note the design: real clinical events are gradual and show up in more than one vital; sensor faults are abrupt, isolated or physically impossible. The detector uses exactly this distinction.

9. Outlier Detection

Layer	Method	Catches
1. Rules	Physiological limits and stuck-value check	Impossible values, dropouts, frozen sensors
2. Univariate	Rolling MAD modified z-score per patient and metric (window 60, flag |z| > 3.5)	Spikes, firmware offsets, sudden shifts
3. Multivariate	River Half-Space Trees on scaled vitals per patient group	Unusual combinations no single metric reveals
Classifier	Rate of change and cross-signal agreement over the last 5 readings	Separates clinical events from sensor errors

```
def robust_z(x, hist):
    med = np.median(hist)
    mad = np.median(np.abs(hist - med)) or 1e-6  # guard flat windows
    return 0.6745 * (x - med) / mad
```

9.1 Evaluation

- Precision, recall and F1 per anomaly type and per layer, plus a combined confusion matrix for sensor_error vs clinical.
- Threshold sweep (2.5 to 5.0) with a precision-recall curve to justify 3.5.
- Detection delay for gradual anomalies (readings between onset and first alert).
- Ablation: rules only, rules + MAD, all three layers.
- Actionable insight: alert rate by firmware version, exposing the faulty v2.3 fleet.

10. Online Machine Learning

10.1 Task

Forecast each patient's glucose **15 minutes ahead**. This is the standard horizon in CGM research and it creates a genuine streaming ML problem: the true value arrives 15 minutes after the prediction (delayed labels). Workers keep a per-patient queue of pending feature vectors and only score and train when the matching label arrives.

10.2 Features

Group	Features
Recent glucose	Current value, lags 1 to 3, slope and acceleration over the last 10 min, rolling mean and sd
Other vitals	Heart rate, systolic BP
Time	Hour of day as sin and cos, minutes since last detected meal rise
Patient context	Condition, on_insulin, on_metformin (cached from dimension tables)

10.3 Models compared

- **Persistence** baseline and **linear trend extrapolation** baseline.
- **River LinearRegression** with StandardScaler.
- **Hoeffding Adaptive Tree Regressor**.
- **Adaptive Random Forest Regressor** (ARF), with built-in drift detection per tree.
- **Population vs personalised**: one global model vs global model plus a small per-patient online correction. This is a research-quality question with a clear answer we can measure.

10.4 Evaluation

- Prequential (test then train) MAE and RMSE over time, 3 random seeds, reported with mean and spread.
- **Clarke Error Grid**: share of predictions in zones A to E. This is the clinical standard for glucose prediction and shows domain depth.
- Hypoglycemia early warning: precision, recall and median lead time when forecast < 70 mg/dL.
- Throughput: predictions per second per worker.

10.5 Concept drift experiment

Mid-run, 20% of diabetic patients switch to a higher insulin dose. We plot error before and after, mark the ADWIN detection point on the error stream, and report detection delay and recovery time for each model. ARF and the adaptive tree should recover faster than plain linear regression, which gives the report a clear, evidence-based conclusion about why online learning matters.

11. Dashboard

Streamlit 1.63 multipage app. Each live panel is a fragment with run_every="5s" and parallel=True, so slow panels never block fast ones. Heavy views read vitals_1min, not raw readings.

Page	Contents
Live Ops	Rows per minute, active patients, open high alerts, end-to-end latency p95, current MAE
Patient	Glucose, BP and HR with alert markers, 15-min forecast band, medication timeline
Alerts	Live feed by type and severity, alert rate by firmware version
Models	Rolling MAE per model, Clarke grid scatter, drift events
System Health	Worker lag, dead-letter rate, duplicate rate, partition sizes, top queries from pg_stat_statements

```
@st.fragment(run_every="5s", parallel=True)
def ingest_kpi():
    n = q("SELECT count(*) FROM vitals_readings WHERE ingested_at > now() - interval '1 minute'")
    st.metric("Rows last minute", f"{n:,}")  # reruns alone, rest of page untouched
```

12. Benchmarks and Engineering Quality

12.1 Benchmark protocol

Experiment	Method	Output
Step load	1k, 5k, 20k, 50k, 100k rows/min, 10 min each, 2 tiers	Throughput vs p95 latency curve, saturation point
Write path	Row INSERT vs executemany vs COPY vs COPY + staging	Rows/s bar chart
Worker scaling	1, 2, 4 workers at fixed load	Lag and throughput vs workers
Index choice	BRIN vs B-tree on recorded_at	Index size, query time
Partitioning	Same queries on partitioned vs plain table at 20M rows	EXPLAIN (ANALYZE, BUFFERS) with pruning
Dashboard	Raw vs rollup queries	p95 query time
Cost	Azure cost per session / rows processed	Cost per 1M readings

Run each experiment 3 times and report median and range. Write results to a benchmark_runs CSV so every chart in the report is generated by a script.

12.2 Engineering practices

- **One command**: make up starts local PostgreSQL 18, seeds data, and runs generators, workers and dashboard via Docker Compose. make azure provisions the cloud server.
- **Config** in environment variables, typed with pydantic-settings. No hard-coded credentials.
- **Tests**: pytest for generator determinism, MAD maths, idempotent writes and the restart test. GitHub Actions runs them against a PostgreSQL 18 service container.
- **Code quality**: ruff for lint and format, type hints, short functions.
- **Structured logs** (JSON) with batch size, latency and lag, feeding the System Health page.

```
carevault-stream/
  Makefile  docker-compose.yml  pyproject.toml  uv.lock  .env.example
  infra/azure_provision.sh      sql/schema.sql  sql/partman.sql
  generator/  sim.py ingest.py            processor/ worker.py outliers.py models.py
  dashboard/  app.py pages/               bench/     step_load.py queries.py
  eval/       outliers_eval.py model_eval.py clarke.py
  tests/                                  report/    figures/ (all script-generated)
```

13. Report and Video

13.1 Report outline (10 to 14 pages)

Section	Content
1. Introduction	CareVault context, problem, contributions list
2. Architecture	Diagram, PostgreSQL-as-log design, why no Kafka/Spark with our numbers
3. Data model	ER diagram, partitioning, BRIN, idempotency, retention
4. Ingestion and scale	Write path comparison, step-load curve, saturation analysis, cost
5. Processing semantics	Visibility-gap problem, effectively-once design, restart test
6. Outlier detection	Layers, threshold sweep, per-type results, ablation, firmware insight
7. Online ML	Delayed labels, models, prequential results, Clarke grid, drift experiment
8. Dashboard	Pages, refresh design, query performance
9. Limitations and future work	Synthetic data, single node, Kafka crossover point, CareVault roadmap
10. Contributions	Who did what

13.2 Video (5 minutes)

Time	Segment	On screen
0:00 to 0:25	Hook and problem	One patient's glucose crashing, then the 2,000-patient view
0:25 to 1:05	Architecture and data model	Diagram, ER diagram, Azure portal
1:05 to 2:00	Scale	Ramp to 50k rows/min live, then the saturation curve
2:00 to 2:50	Outliers	Firmware fault appearing on dashboard, per-type F1 table
2:50 to 3:50	Online ML	Forecast band on patient page, drift spike and recovery, Clarke grid
3:50 to 4:20	Reliability	Kill a worker, restart, reconciliation shows zero loss
4:20 to 5:00	Results and close	Key numbers slide, limitations, future work

Record each segment separately and edit together. Keep a backup recording of every live demo.

14. Timeline (5 weeks)

Priority tiers. MUST items satisfy the brief. SHOULD items make the project excellent. STRETCH items only start once every MUST and SHOULD is done. A finished, measured project beats an ambitious unfinished one.

Week	MUST	SHOULD / STRETCH
1	Azure PG 18 server, extensions, roles. Repo, uv, Docker Compose, CI. Schema with partitions. Seed script.	Provisioning script, TimescaleDB availability check

Week	MUST	SHOULD / STRETCH
2	Generator with physiology and anomalies. COPY + staging ingest. Dedupe and dead_letter. Baseline throughput.	Parallel generators, messy-data cases, write-path benchmark
3	Worker with effectively-once commits. Rules + MAD. Dashboard Live Ops and Alerts. Rollup via pg_cron.	Half-Space Trees, worker scaling, restart test
4	River models with delayed labels, prequential MAE. Outlier evaluation. Patient and Models pages.	ARF, personalised model, Clarke grid, drift experiment
5	Backfill to 20M rows, final benchmark runs, report, video.	System Health page polish, cost analysis

15. Team Roles

Member	Owns	Report sections
A: Data platform	Azure, schema, partitioning, indexes, rollups, query tuning, cost	3, and DB half of 4
B: Ingestion	Generator, messy data, write path, step-load harness	4
C: Processing and detection	Workers, effectively-once design, outlier layers and evaluation	5, 6
D: ML and dashboard	Online models, drift, Clarke grid, Streamlit app, video edit	7, 8

Weekly: each member reviews one other member's pull request. Twice-weekly 20-minute sync. Decisions go in docs/decisions.md with a one-line reason each; this becomes the design-choices part of the report.

16. Risks

Risk	Mitigation
Azure credit runs out	B2s for dev, scale up only for timed benchmark sessions, stop server when idle, budget alert
pg_partman or pg_cron setup issues on Azure	Day 1 task. Fallback: create daily partitions from Python at startup
Virtual generated column fails on partitions	Move MAP into a view
Benchmarks limited by laptop, not database	Run generators on 2 machines or a small Azure VM in the same region
Model does not beat baseline	Check lag features and label alignment first; the trend baseline is hard to beat, report it honestly
Scope creep	Tiered timeline; nothing STRETCH before MUST and SHOULD are done
Demo fails while recording	Segmented recording, local Docker fallback with identical schema

17. Final Checklist

- Schema on Azure PostgreSQL 18 with partitions, BRIN and FKs; screenshot captured
- Step-load curve with saturation point and cost per 1M readings
- 20M+ readings at final run; table and index sizes recorded
- Restart test passes; reconciliation query shows zero loss and zero duplicates
- Outlier P/R/F1 per type, threshold sweep, ablation, firmware insight
- Prequential MAE vs baselines over 3 seeds, Clarke grid, drift detection and recovery
- Dashboard: all five pages live, p95 query time under target
- One-command setup verified on a fresh clone; CI green; no secrets in git
- Report with contributions; video under 5 minutes